Assignment 06

Digital RTL Design

CDC & RDC

Marwan Abdelfattah

Contents

1 FIFO Recall **2**
1.1 Original FIFO Implementation 2
1.2 CDC Analysis of the FIFO 3

2 Question 1 **4**
2.1 Problem Statement 4
2.2 RTL Design 5

3 Question 2 **6**
3.1 Problem Statement 6
3.2 RTL Design 7
3.2.1 Binary-to-Gray Converter 7
3.2.2 Gray-to-Binary Converter 7
3.2.3 Gray Encoded Synchronizer 8

4 Question 3 **9**
4.1 Problem Statement 9
4.2 RTL Design 10

5 Conclusion **11**

6 Question 4 **12**
6.1 DSP48A1 Reset Domain Crossing (RDC) Verification 12
6.1.1 RDC Analysis Before Synchronization 12
6.1.2 RDC Analysis After Synchronization 14

1 FIFO Recall

In Assignment 5, a First-In-First-Out (FIFO) memory was implemented to store and retrieve data in the same order in which it was received. The FIFO supports two main operations: a **write operation**, controlled by `clk_a`, and a **read operation**, controlled by `clk_b`.

Two internal pointers are used to control the FIFO operation:

- **Write Pointer (`wr_ptr`):** Indicates the memory location where the next input data will be written. The write pointer is updated in the `clk_a` clock domain.
- **Read Pointer (`rd_ptr`):** Indicates the memory location from which the next data will be read. The read pointer is updated in the `clk_b` clock domain.

The `full` flag is used to prevent writing new data when the FIFO is full, while the `empty` flag prevents reading data when the FIFO is empty.

1.1 Original FIFO Implementation

The following Verilog RTL implementation was developed in Assignment 5.

```
module FIFO(din_a, wen_a, ren_b, clk_a, clk_b,
            rst, dout_b, full, empty);

    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 512;
    parameter ADDR_SIZE  = 9;

    input [FIFO_WIDTH-1:0] din_a;
    input wen_a, ren_b, clk_a, clk_b, rst;

    output reg [FIFO_WIDTH-1:0] dout_b;
    output full, empty;

    reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];
    reg [ADDR_SIZE:0] wr_ptr, rd_ptr;

    // Write operation
    always @(posedge clk_a) begin
        if (rst) begin
            wr_ptr <= 0;
        end
        else if (wen_a && full != 1) begin
            mem[wr_ptr[ADDR_SIZE-1:0]] <= din_a;
            wr_ptr <= wr_ptr + 1;
        end
    end

    // Read operation
    always @(posedge clk_b) begin
        if (rst) begin
            rd_ptr <= 0;
```

```
        end
        else if (ren_b && empty != 1) begin
            dout_b <= mem[rd_ptr[ADDR_SIZE-1:0]];
            rd_ptr <= rd_ptr + 1;
        end
    end

    // Full and empty flag generation
    assign full =
        ((wr_ptr[ADDR_SIZE] != rd_ptr[ADDR_SIZE]) &&
         (wr_ptr[ADDR_SIZE-1:0] ==
          rd_ptr[ADDR_SIZE-1:0])) ? 1 : 0;

    assign empty = (wr_ptr == rd_ptr) ? 1 : 0;

endmodule
```

1.2 CDC Analysis of the FIFO

Although the original FIFO operates correctly in functional simulation, running **Questa CDC** reveals Clock Domain Crossing (CDC) violations.

The reason is that the FIFO contains two different clock domains:

$$\text{Write Domain} = \texttt{clk_a}$$

$$\text{Read Domain} = \texttt{clk_b}.$$

The write pointer `wr_ptr` is generated in the `clk_a` domain, while the read pointer `rd_ptr` is generated in the `clk_b` domain. However, the original implementation directly uses both pointers when generating the `full` and `empty` flags.

For the `full` condition, the read pointer generated in the `clk_b` domain is directly used by logic controlling the `clk_a` domain. Therefore, the following CDC path exists:

$$\texttt{rd_ptr} \quad (\texttt{clk_b}) \rightarrow \texttt{full} \rightarrow \texttt{clk_a}.$$

Similarly, for the `empty` condition, the write pointer generated in the `clk_a` domain is directly used by logic controlling the `clk_b` domain:

$$\texttt{wr_ptr} \quad (\texttt{clk_a}) \rightarrow \texttt{empty} \rightarrow \texttt{clk_b}.$$

Since `wr_ptr` and `rd_ptr` are multi-bit binary counters, directly transferring them between asynchronous clock domains may result in metastability or the destination domain observing an inconsistent pointer value.

When the design was analyzed using **Questa CDC**, multiple-bit clock domain crossing violations were detected between the write and read clock domains. A FIFO memory synchronization caution was also reported because the FIFO memory is written using `clk_a` and read using `clk_b`.

Therefore, the original FIFO architecture must be modified so that pointer information is properly synchronized before it is used in the opposite clock domain. The following

question introduces the required modifications using the Gray-Encoded Synchronizer developed previously.

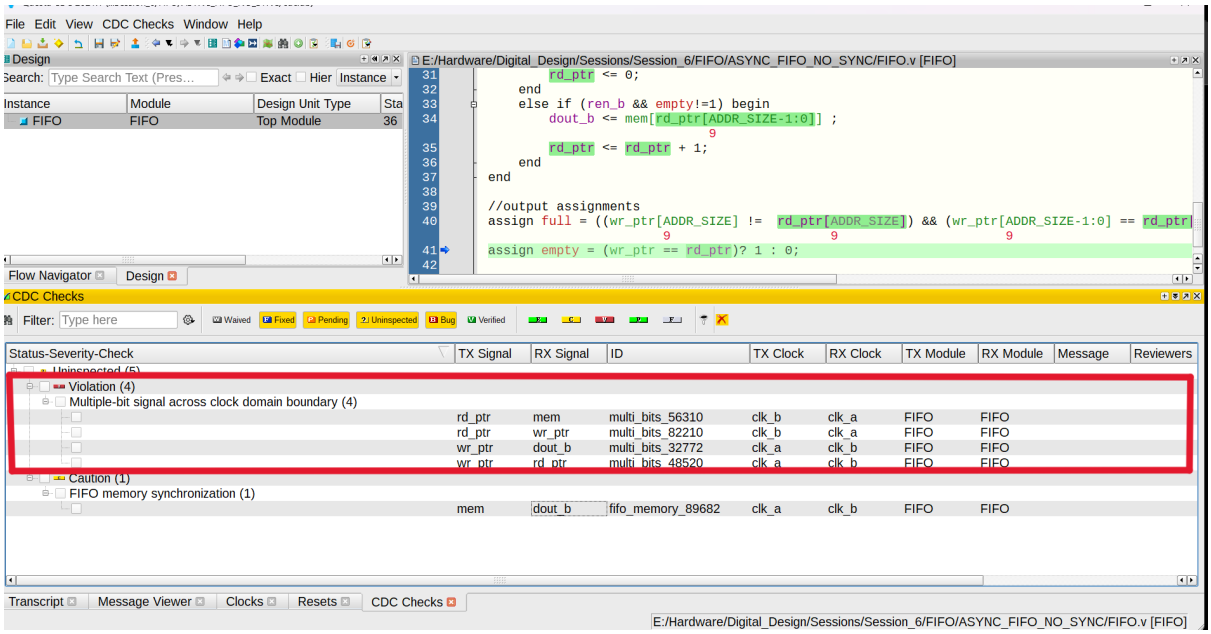


Figure 1: 2 DFF Synchronizers

2 Question 1

2.1 Problem Statement

Create a Verilog design for a **Two-DFF Synchronizer**. The synchronizer shall consist of two flip-flops connected in series and shall support configurable data width and reset type. The design shall have the following parameters:

- `WIDTH`: Determines the width of the input `in` and output `out`. The default value is 1.
- `RSTTYPE`: Determines whether the reset of the synchronizer is synchronous (`SYNC`) or asynchronous (`ASYNC`). The default value is `SYNC`.

The design shall have the following ports:

- `clk`: Clock input.
- `rst`: Reset input.
- `in`: Input signal with a width determined by the `WIDTH` parameter.
- `out`: Synchronized output signal with a width determined by the `WIDTH` parameter.

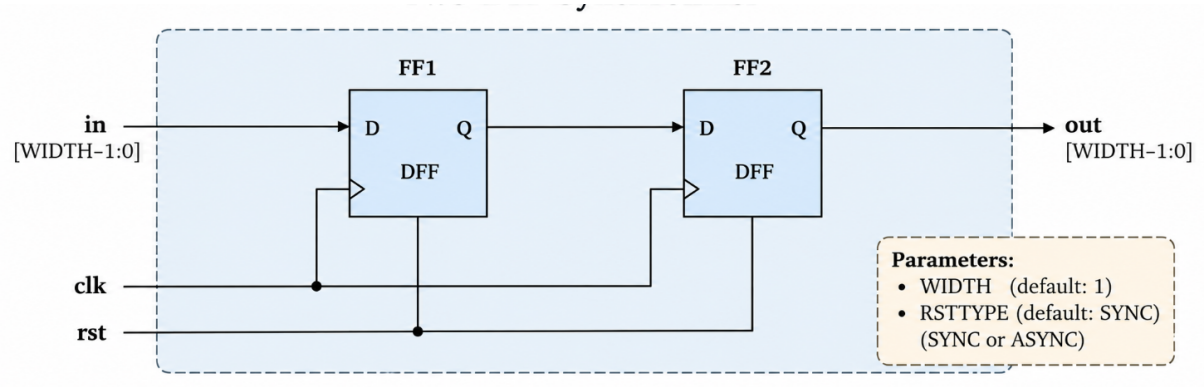


Figure 2: 2 DFF Synchronizers

2.2 RTL Design

The following Verilog RTL implementation was developed according to the requirements of Question 1.

```
module DFF2_sync(in, out, clk, rst);
    parameter WIDTH = 1;
    parameter RSTTYPE = "SYNC";
    input clk, rst;
    input [WIDTH-1:0] in;
    output reg [WIDTH-1:0] out;
    reg [WIDTH-1:0] q;

     generate
        if(RSTTYPE== "SYNC") begin
            always @(posedge clk) begin
                if(rst)begin
                    q <= 0;
                    out <= 0;
                end else begin
                    q <= in;
                    out <= q;
                end
            end
        end
        else if (RSTTYPE== "ASYNC") begin
            always @(posedge clk or posedge rst) begin
                if(rst)begin
                    q <= 0;
                    out <= 0;
                end else begin
                    q <= in;
                    out <= q;
                end
            end
        end
    endgenerate
endmodule
```

3 Question 2

3.1 Problem Statement

Create a Verilog design for a **Gray Encoded Synchronizer**. The design shall instantiate the Two-DFF Synchronizer implemented in the previous question. In addition, separate **Binary-to-Gray** and **Gray-to-Binary** conversion modules shall be created and instantiated within the Gray Encoded Synchronizer.

The design shall have the following parameters:

- `WIDTH`: Determines the width of the input `tx_data` and output `rx_data`. The default value is 8.
- `RSTTYPE`: Determines whether the reset of the registers is synchronous (`SYNC`) or asynchronous (`ASYNC`). The default value is `SYNC`.

The design shall have the following ports:

- `tx_data`: Input data signal with a width determined by the `WIDTH` parameter.
- `rx_data`: Output data signal with a width determined by the `WIDTH` parameter.
- `tx_clk`: Transmit clock input.
- `rx_clk`: Receive clock input.
- `rst`: Reset input signal.

Binary-to-Gray Conversion

Given a binary number $B[n-1:0]$, the corresponding Gray code $G[n-1:0]$ shall be calculated as follows:

- The most significant bit (MSB) remains unchanged:

$$G[n-1] = B[n-1]$$

- For all remaining bits, from $n-2$ down to 0:

$$G[i] = B[i+1] \oplus B[i]$$

Gray-to-Binary Conversion

Given a Gray code input $G[n-1:0]$, the corresponding binary output $B[n-1:0]$ shall be calculated as follows:

- The most significant bit (MSB) remains unchanged:

$$B[n-1] = G[n-1]$$

- For all remaining bits, from $n-2$ down to 0:

$$B[i] = B[i+1] \oplus G[i]$$

The Binary-to-Gray and Gray-to-Binary conversion logic shall be implemented using the Verilog `generate-for` construct.

The Gray Encoded Synchronizer shall convert the binary data from the transmit clock domain into Gray code, synchronize the Gray-encoded data into the receive clock domain using the Two-DFF Synchronizer, and finally convert the synchronized Gray code back into binary form.

3.2 RTL Design

3.2.1 Binary-to-Gray Converter

The following Verilog RTL implementation converts the binary input data into its corresponding Gray code representation.

```
module binary_to_gray(binary,gray);
    parameter WIDTH = 8;
    input [WIDTH-1:0] binary;
    output [WIDTH-1:0] gray;

    genvar i;
    generate
        for (i=WIDTH-2;i>=0;i=i-1) begin
            assign gray[i] = binary[i+1] ^ binary[i];
        end
    endgenerate

    assign gray[WIDTH-1] = binary[WIDTH-1];
endmodule
```

3.2.2 Gray-to-Binary Converter

The following Verilog RTL implementation converts the Gray encoded input data back into its corresponding binary representation.

```
module gray_to_binary(gray, binary);
    parameter WIDTH = 8;
    input [WIDTH-1:0] gray;
    output [WIDTH-1:0] binary;

    genvar i;
    generate
        for (i=WIDTH-2;i>=0;i=i-1) begin
            assign binary[i] = binary[i+1] ^ gray[i];
        end
    endgenerate

    assign binary[WIDTH-1] = gray[WIDTH-1];
endmodule
```

3.2.3 Gray Encoded Synchronizer

The following Verilog RTL implementation represents the complete Gray Encoded Synchronizer. It instantiates the Binary-to-Gray Converter, the Two-DFF Synchronizer developed in Question 1, and the Gray-to-Binary Converter.

```
module gray_enc_sync(tx_data, rx_data, tx_clk, rx_clk, rst);
    parameter WIDTH = 8;
    parameter RSTTYPE = "SYNC";
    input tx_clk, rx_clk, rst;
    input [WIDTH-1:0] tx_data;
    output [WIDTH-1:0] rx_data;


    wire [WIDTH-1:0] gray, gray_sync;
    reg  [WIDTH-1:0] gray_reg;

    binary_to_gray #(.WIDTH(WIDTH)) bg1 (tx_data, gray);
    DFF2_sync #(.WIDTH(WIDTH),.RSTTYPE(RSTTYPE)) sync (gray_reg, gray_sync,
   rx_clk, rst);
    gray_to_binary #(.WIDTH(WIDTH)) gb1(gray_sync, rx_data);

        generate
        if(RSTTYPE== "SYNC") begin
            always @(posedge tx_clk) begin
                if(rst)begin
                    gray_reg <= 0;
                end else begin
                    gray_reg <= gray;
                end
            end
        end
        else if (RSTTYPE== "ASYNC") begin
            always @(posedge tx_clk or posedge rst) begin
                if(rst)begin
                    gray_reg <= 0;
                end else begin
                    gray_reg <= gray;
                end
            end
        end
    endgenerate


endmodule
```

4 Question 3

4.1 Problem Statement

Modify the FIFO design implemented in the previous assignment. Since writing to the FIFO occurs in the `clk_a` clock domain and reading from the FIFO occurs in the `clk_b` clock domain, Clock Domain Crossing (CDC) issues and metastability may occur. Therefore, the `full` and `empty` flag logic must be implemented using proper synchronization.

The FIFO design shall be modified according to the following requirements:

- The `full` flag is determined in the write clock domain (`clk_a`). Therefore, the read pointer must first be synchronized into the `clk_a` domain before being used in the `full` flag logic.
- The `empty` flag is determined in the read clock domain (`clk_b`). Therefore, the write pointer must first be synchronized into the `clk_b` domain before being used in the `empty` flag logic.
- The Gray-Encoded Synchronizer implemented in Question 2 shall be instantiated inside the FIFO to synchronize the read and write pointers between the two clock domains.
- The `full` and `empty` flag logic shall be modified to use the synchronized pointer values instead of directly using pointers from the opposite clock domain.
- The modified FIFO design shall be analyzed using **Questa CDC** to verify that no Clock Domain Crossing violations remain.

The required pointer synchronization paths are therefore:

`rd_ptr` → Gray-Encoded Synchronizer → `clk_a domain` → `full`

and

`wr_ptr` → Gray-Encoded Synchronizer → `clk_b domain` → `empty`.

4.2 RTL Design

The following Verilog RTL implementation was developed according to the requirements of Synchronized FIFO.

```
module FIFO(din_a, wen_a, ren_b, clk_a ,clk_b, rst, dout_b, full, empty);
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 512;
parameter ADDR_SIZE = 9 ;

input [FIFO_WIDTH-1:0] din_a;
input wen_a, ren_b, clk_a, clk_b, rst;
output  reg [FIFO_WIDTH-1:0] dout_b;
output full, empty;

reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];
reg [ADDR_SIZE:0] wr_ptr, wr_ptr_sync, rd_ptr, rd_ptr_sync;

gray_enc_sync #(.WIDTH(ADDR_SIZE),.RSTTYPE("SYNC")) rd_ptr_sync_inst(rd_ptr,
    rd_ptr_sync, clk_b, clk_a, rst);
gray_enc_sync #(.WIDTH(ADDR_SIZE),.RSTTYPE("SYNC")) wr_ptr_sync_inst(wr_ptr,
    wr_ptr_sync, clk_a, clk_b, rst);

//write operation
always @(posedge clk_a) begin
    if(rst)begin
        wr_ptr <= 0;
    end
    else if (wen_a && full!=1) begin
        mem[wr_ptr[ADDR_SIZE-1:0]] <= din_a;
        wr_ptr <= wr_ptr + 1;
    end
end

//read operation
always @(posedge clk_b) begin
    if(rst)begin
        rd_ptr <= 0;
    end
    else if (ren_b && empty!=1) begin
        dout_b <= mem[rd_ptr[ADDR_SIZE-1:0]] ;
        rd_ptr <= rd_ptr + 1;
    end
end

//output assignments
assign full = ((wr_ptr[ADDR_SIZE] !=  rd_ptr_sync[ADDR_SIZE]) && (wr_ptr[
    ADDR_SIZE-1:0] == rd_ptr_sync[ADDR_SIZE-1:0]))? 1 : 0;
assign empty = (wr_ptr_sync == rd_ptr)? 1 : 0;


endmodule
```

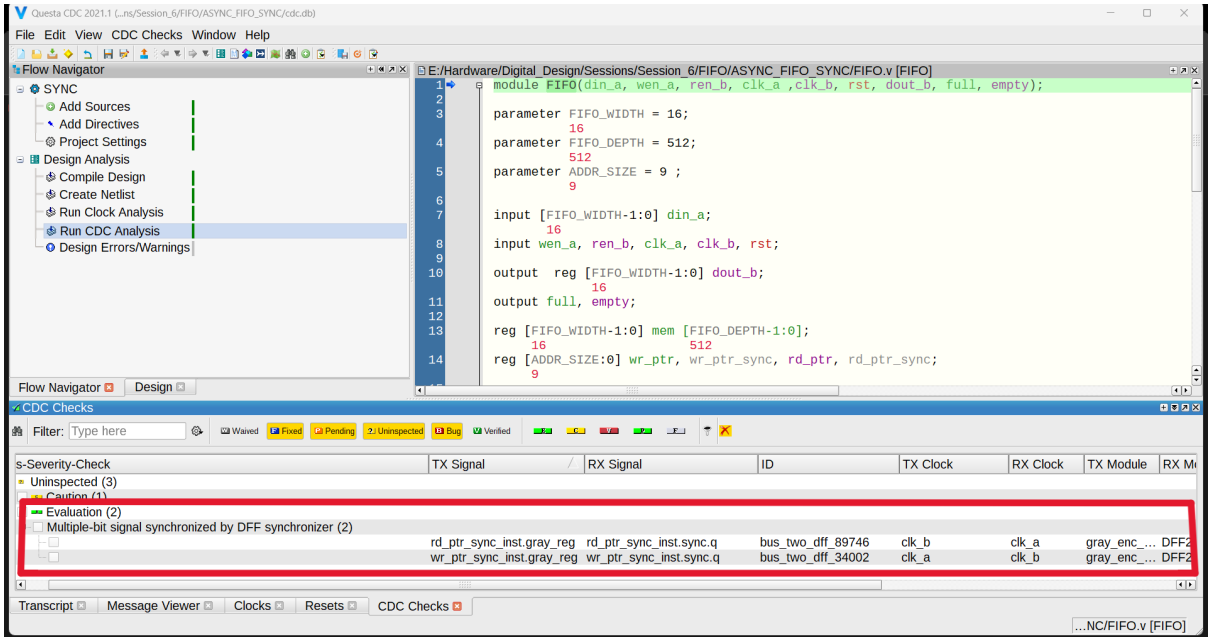


Figure 3: Questa CDC Snippet

5 Conclusion

In this assignment, Clock Domain Crossing (CDC) concepts and synchronization techniques were investigated and implemented using Verilog. A parameterized Two-DFF synchronizer was first designed to provide a basic synchronization mechanism between different clock domains. A Gray-encoded synchronizer was then implemented using Binary-to-Gray and Gray-to-Binary conversion modules together with the Two-DFF synchronizer.

These synchronization techniques were subsequently applied to the FIFO design developed in the previous assignment. The original FIFO contained unsafe crossings because the write and read pointers were directly used across the `clk_a` and `clk_b` clock domains. The design was modified by synchronizing the pointer information before using it to determine the `full` and `empty` conditions.

All circuit designs were implemented in Verilog. In addition, Questa CDC was used to analyze the clock domain crossings and verify the synchronization structures in the modified FIFO design. Overall, the assignment demonstrated the importance of proper CDC handling and the use of synchronization techniques when transferring signals between asynchronous clock domains.

6 Question 4

6.1 DSP48A1 Reset Domain Crossing (RDC) Verification

Modify the DSP48A1 design developed in Project 1 to prevent Reset Domain Crossing (RDC) violations when the DSP block registers use asynchronous resets.

Design Requirements:

- Instantiate the synchronizer developed in Question 1 between the appropriate registers in the DSP48A1 block to ensure safe reset domain transitions.
- Set the `RSTTYPE` parameter of the DSP48A1 block to `"ASYNC"` to verify asynchronous reset behavior.
- Use a `generate-if` construct to conditionally instantiate the synchronizers only when `RSTTYPE` is set to `"ASYNC"`.
- Run **Questa RDC** analysis to verify that the modified DSP48A1 design has no Reset Domain Crossing violations.

Reference Design:

The original DSP48A1 RTL implementation from Project 1 is available in the `Digital_Design_Verification_Projects` GitHub repository, under the `DSP48A1` directory.

Verification Objective:

Ensure that the modified DSP48A1 architecture supports asynchronous reset operation while maintaining safe reset domain transitions and achieving zero RDC violations in Questa RDC.

6.1.1 RDC Analysis Before Synchronization

Before implementing the synchronizers, the original DSP48A1 design was analyzed using **Questa RDC** with the `RSTTYPE` parameter configured as `"ASYNC"`.

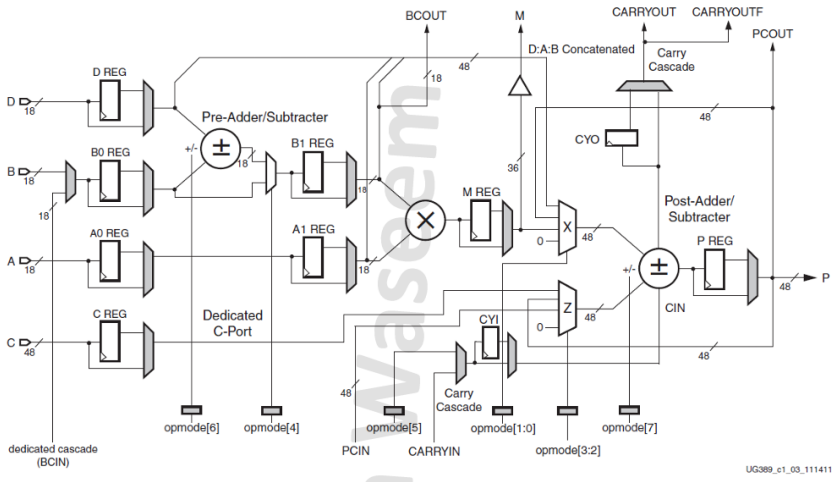


Figure 4: DSP48A1 Before Synchronizers design showing 19 potential RDC violations before synchronization.

The initial analysis identified **19 potential Reset Domain Crossing (RDC) violations** between registers controlled by different asynchronous reset signals, despite operating under the same clock (`CLK`).

The reported violations involved the following registers:

- `OPMODE` register
- `A1_reg`
- `B1_reg`
- `C_reg`
- `D_reg`
- `M_reg`
- `P_reg`
- `CARRYOUT` register

The violations were primarily detected along paths feeding the multiplier register (`M_reg`), output register (`P_reg`), carry-out register, and intermediate pipeline registers.

These crossings arise because the source and destination registers are controlled by independent asynchronous reset signals, potentially allowing unsafe data transitions during reset assertion or deassertion.

Figure 5 illustrates the RDC violations reported by Questa RDC before applying synchronization.

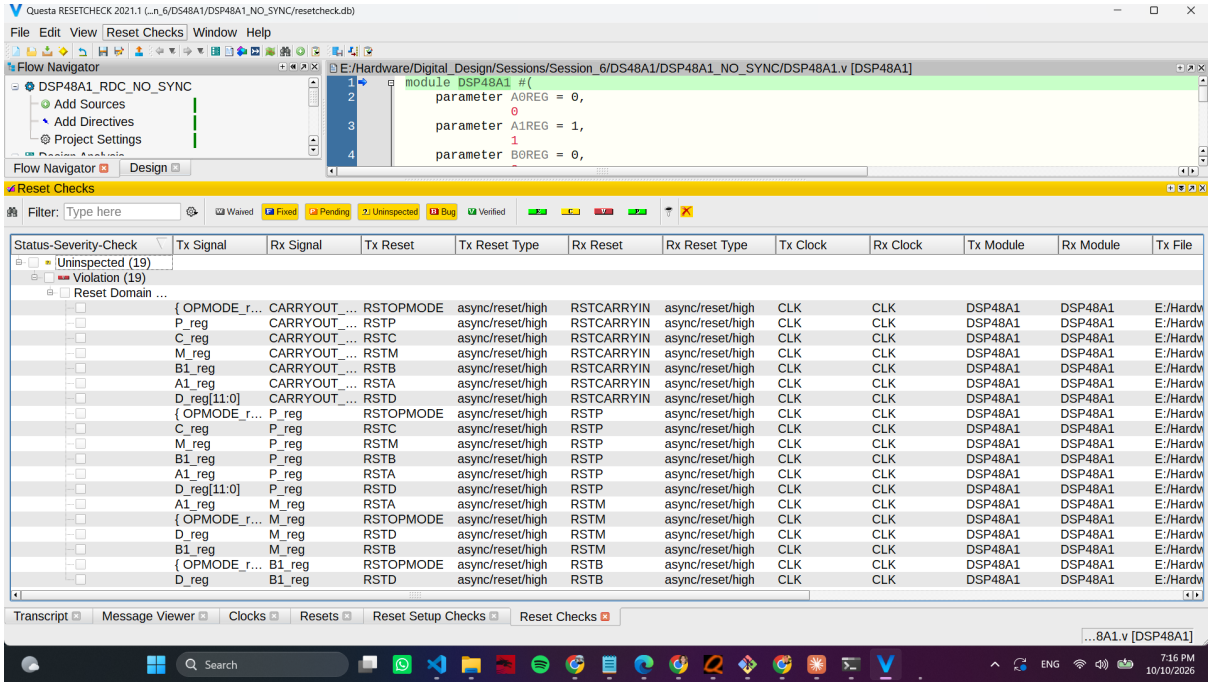


Figure 5: Questa RDC analysis of the original DSP48A1 design showing 19 potential RDC violations before synchronization.

To address these violations, synchronizers will be conditionally instantiated using `generate-if` when `RSTTYPE = "ASYNC"`.

The modified design will then be reanalyzed using **Questa RDC** to verify that the reported reset domain crossing violations have been resolved.

6.1.2 RDC Analysis After Synchronization

After identifying the 19 potential Reset Domain Crossing (RDC) violations in the original DSP48A1 design, the RTL was modified by introducing synchronizers at the appropriate locations between registers belonging to different asynchronous reset domains.

The synchronizers were conditionally instantiated using `generate-if` constructs when the `RSTTYPE` parameter was set to `"ASYNC"`, while preserving the original datapath behavior for synchronous reset operation.

Synchronizer Placement:

Figure 6 illustrates the modified DSP48A1 architecture. The small red rectangles indicate the locations where synchronizers were instantiated to address the identified reset domain crossings.

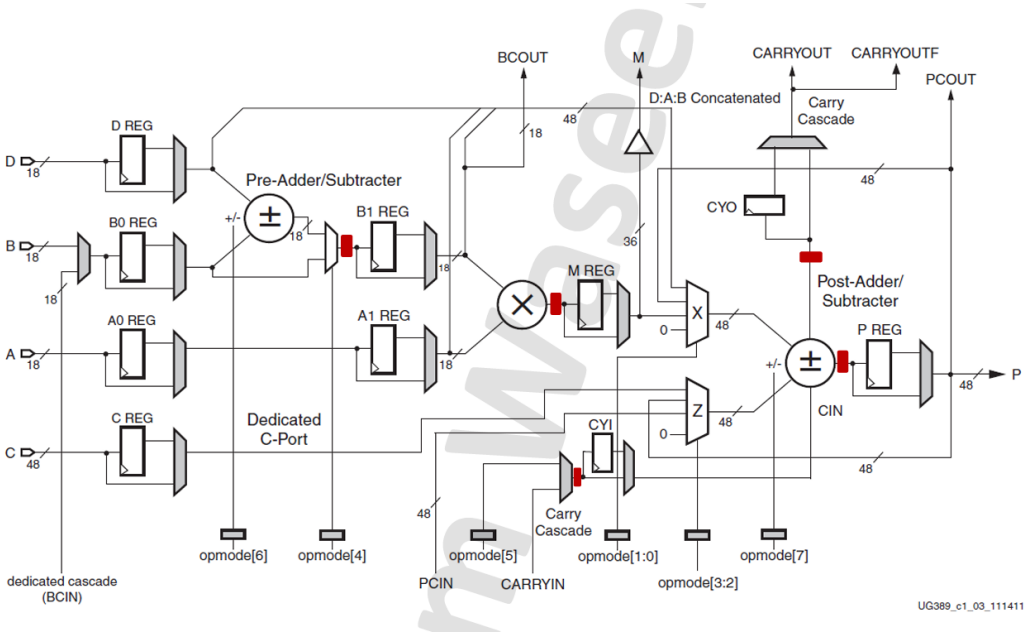


Figure 6: DSP48A1 architecture highlighting the locations of the instantiated synchronizers (red rectangles).

Questa RDC Verification Results:

Following the modifications, the DSP48A1 design was reanalyzed using **Questa RDC** to evaluate the effectiveness of the inserted synchronizers.

As illustrated in Figure 7, the original 19 RDC violations were no longer reported as unsynchronized crossing violations.

Instead, Questa RDC identified 20 synchronization-related checks under the `Evaluation` category:

- **19 checks:** `Multiple-bit RDC Signal Synchronized By DFF Synchronizer`.
- **1 check:** `RDC Signal Synchronized By DFF Synchronizer`.

These results indicate that the previously reported unsynchronized reset domain crossings are now recognized as paths containing DFF-based synchronizers.

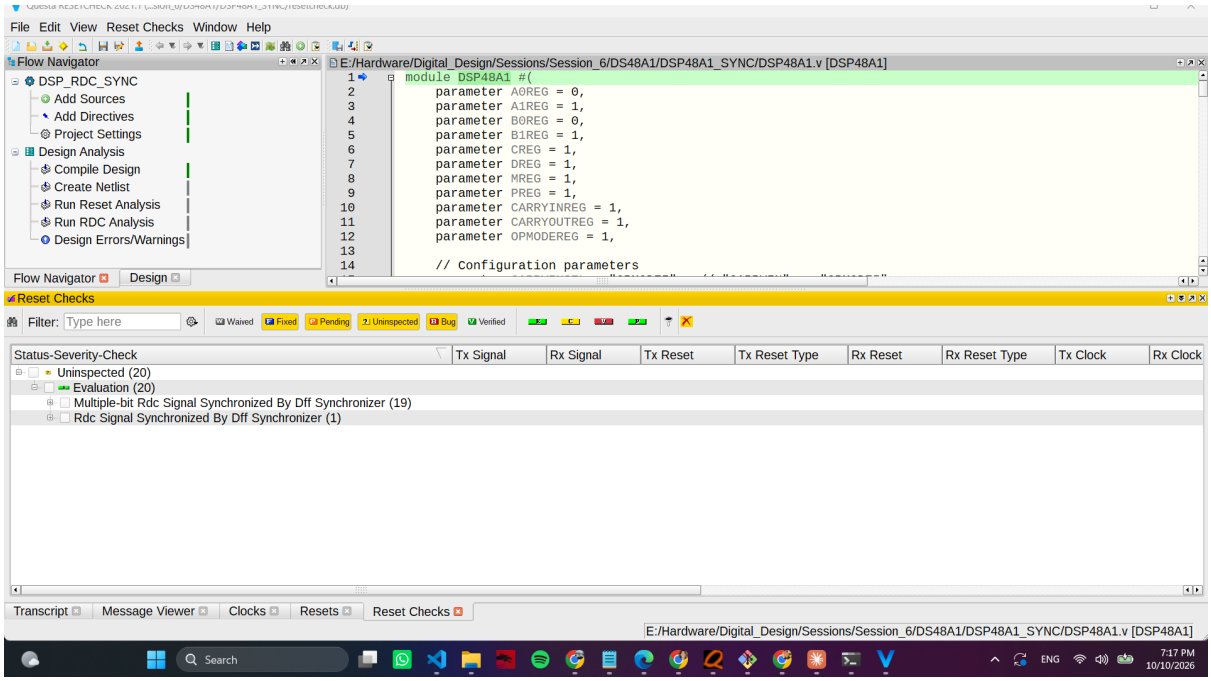


Figure 7: Questa RDC analysis after synchronization, showing 20 synchronization-related evaluation checks instead of the original unsynchronized RDC violations.

Conclusion:

The RDC analysis demonstrates that the inserted synchronizers successfully changed the classification of the original reset domain crossings from unsynchronized violations to synchronized paths.

The final Questa RDC report contains 20 evaluation checks associated with the implemented synchronizers, with no unsynchronized RDC violations shown in the displayed results.

These evaluation checks require further review, particularly for multi-bit signals, to confirm that the synchronization strategy is functionally safe and that no data-coherency issues are introduced.